NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....
.....
.....

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

.....
.....
.....

LỜI CẢM ƠN

Trước tiên, em xin chân thành gửi lời cảm ơn sâu sắc đến Ban Giám hiệu Nhà trường, các thầy cô trong Khoa Kỹ thuật và Công nghệ đã tạo điều kiện thuận lợi cho em trong suốt quá trình học tập và thực hiện đồ án.

Em xin bày tỏ lòng biết ơn chân thành đến thầy TS. Nguyễn Văn A – giảng viên hướng dẫn trực tiếp – đã tận tình hướng dẫn, chỉ bảo và động viên em trong suốt quá trình thực hiện đồ án này.

Do khả năng và thời gian có hạn, đồ án không tránh khỏi những thiếu sót. Em kính mong nhận được sự đóng góp ý kiến từ các thầy cô và các bạn để đồ án được hoàn thiện hơn.

MỤC LỤC

CHƯƠNG 1: NGHIÊN CỨU LÝ THUYẾT 11
1.1. ASP.NET Core MVC 11
1.2. Entity Framework Core 11
1.2.1. Code-First Approach 11
1.2.2. Migration 12
1.3. SQL Server LocalDB 12
1.4. Bootstrap 5 và Font Awesome 12
CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU 13
2.1. Mô tả bài toán 13
2.2. Yêu cầu chức năng 14
2.3. Mô hình cơ sở dữ liệu 14
2.4. Kiến trúc hệ thống 16
CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU 17
3.1. Môi trường phát triển và triển khai 17
3.2. Giao diện hệ thống 17
3.3. Chức năng chính đã hiện thực 20
3.4. Kiểm thử hệ thống 21
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 23
4.1. Kết luận 23
4.2. Hướng phát triển 23
DANH MỤC TÀI LIỆU THAM KHẢO 25
A.1. Yêu cầu hệ thống 26
A.2. Các bước triển khai 26
B.1. Program.cs 27
B.2. Models/Doctor.cs 30
B.3. Models/Specialty.cs 30

B.4. Models/Appointment.cs 31
B.5. Models/Schedule.cs 31
B.6. Models/ApplicationUser.cs 32
B.7. Data/ApplicationDbContext.cs 32
B.8. Controllers/HomeController.cs 33
B.9. Controllers/DoctorsController.cs 34
B.10. Controllers/AppointmentsController.cs 34
B.11. Controllers/AccountController.cs 35
B.12. appsettings.json 36
B.13. Views/Shared/_Layout.cshtml 37
B.14. Views/Home/Index.cshtml 38

Lời cảm ơn 3
Mục lục 4
Danh mục hình ảnh / Bảng biểu 5
Tóm tắt đồ án 6
Mở đầu 7
CHƯƠNG 1: NGHIÊN CỨU LÝ THUYẾT 9
1.1. ASP.NET Core MVC 9
1.2. Entity Framework Core 10
1.2.1. Code-First Approach 10
1.2.2. Migration 11
1.3. SQL Server LocalDB 11
1.4. Bootstrap 5 và Font Awesome 11
CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU 12
2.1. Mô tả bài toán 12
2.2. Yêu cầu chức năng 13
2.3. Mô hình cơ sở dữ liệu 14

2.4. Kiến trúc hệ thống 17

CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU 18

3.1. Môi trường phát triển và triển khai 18

3.2. Giao diện hệ thống 19

3.3. Chức năng chính đã hiện thực 22

3.4. Kiểm thử hệ thống 23

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 24

4.1. Kết luận 24

4.2. Hướng phát triển 25

DANH MỤC TÀI LIỆU THAM KHẢO 26

PHỤ LỤC A: HƯỚNG DẪN CÀI ĐẶT 27

PHỤ LỤC B: CODE NGUỒN 28

DANH MỤC HÌNH ẢNH

Hình 3.1: Giao diện trang chủ

Hình 3.2: Giao diện đăng ký tài khoản

Hình 3.3: Giao diện đăng nhập

Hình 3.4: Giao diện danh sách bác sĩ

Hình 3.5: Giao diện chi tiết bác sĩ

Hình 3.6: Giao diện đặt lịch hẹn

Hình 3.7: Giao diện quản lý lịch hẹn

Hình 3.8: Sơ đồ use case hệ thống

Hình 3.9: Kiến trúc hệ thống 3 lớp

DANH MỤC BẢNG BIỂU

Bảng 3.1: Cấu trúc bảng Specialties

Bảng 3.2: Cấu trúc bảng Doctors

Bảng 3.3: Cấu trúc bảng Appointments

Bảng 4.1: Kết quả kiểm thử hệ thống

TÓM TẮT ĐỒ ÁN

Đồ án "Xây dựng website đặt lịch hẹn khám bệnh ngoài giờ" được phát triển nhằm giải quyết các vấn đề trong việc đăng ký khám chữa bệnh truyền thống. Hệ thống cho phép bệnh nhân đặt lịch hẹn trực tuyến mọi lúc, mọi nơi, đồng thời giúp quản lý lịch hẹn một cách hiệu quả.

Công nghệ sử dụng: ASP.NET Core 6.0, Entity Framework Core, SQL Server LocalDB, Bootstrap 5.1.3.

Kết quả đạt được: Hệ thống hoàn chỉnh với các chức năng đăng ký/đăng nhập, tìm kiếm bác sĩ theo chuyên khoa, đặt lịch hẹn trực tuyến và quản lý lịch hẹn cho bệnh nhân.

Từ khóa: ASP.NET Core, Entity Framework, SQL Server, đặt lịch khám bệnh, hệ thống y tế trực tuyến.

MỞ ĐẦU

(Trình bày lý do chọn đề tài, mục tiêu, đối tượng và phạm vi nghiên cứu)

1. Lý do chọn đề tài

Trong bối cảnh phát triển mạnh mẽ của công nghệ thông tin, việc ứng dụng công nghệ vào lĩnh vực y tế đang ngày càng trở nên cấp thiết. Tuy nhiên, hệ thống khám bệnh truyền thống vẫn còn tồn tại nhiều hạn chế như: bệnh nhân mất nhiều thời gian chờ đợi, khó tìm kiếm bác sĩ phù hợp, nhân viên lễ tân bị quá tải trong giờ cao điểm và không có hệ thống quản lý lịch hẹn tập trung.

Xuất phát từ những vấn đề thực tiễn trên, đề tài "Xây dựng website đặt lịch hẹn khám bệnh ngoài giờ" được chọn nhằm ứng dụng công nghệ ASP.NET Core MVC để xây dựng một hệ thống quản lý lịch hẹn khám bệnh trực tuyến hiệu quả, tiện lợi.

2. Mục tiêu nghiên cứu

Mục tiêu tổng quát: Xây dựng website đặt lịch khám bệnh trực tuyến hoàn chỉnh, áp dụng các công nghệ hiện đại trong lập trình web với ASP.NET Core MVC.

Mục tiêu cụ thể:

- Phân tích, thiết kế hệ thống cơ sở dữ liệu phục vụ quản lý lịch hẹn khám bệnh.
- Phát triển các chức năng: đăng ký/đăng nhập, tìm kiếm bác sĩ, đặt lịch hẹn và quản lý lịch hẹn.
- Triển khai, kiểm thử và đánh giá hệ thống hoàn chỉnh.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Hệ thống đặt lịch hẹn khám bệnh với ba nhóm người dùng chính: bệnh nhân, bác sĩ và quản trị viên.

Phạm vi nghiên cứu: Ứng dụng Web trên nền tảng ASP.NET Core MVC, sử dụng SQL Server LocalDB, triển khai trong môi trường phát triển cục bộ.

4. Phương pháp nghiên cứu

- Phân tích và thiết kế hệ thống theo phương pháp hướng đối tượng, sử dụng ngôn ngữ mô hình hóa UML.
- Phát triển phần mềm theo mô hình Agile, linh hoạt điều chỉnh theo yêu cầu thực tế.
- Kiểm thử chức năng (functional testing) và kiểm thử tích hợp (integration testing).

CHƯƠNG 1: NGHIÊN CỨU LÝ THUYẾT

Trình bày cơ sở lý thuyết, lý luận, giả thiết khoa học và phương pháp nghiên cứu đã được sử dụng trong đồ án.

1.1. ASP.NET Core MVC

ASP.NET Core MVC là một framework mã nguồn mở, đa nền tảng và có hiệu suất cao dùng để xây dựng các ứng dụng web hiện đại, được Microsoft phát triển trên nền tảng .NET 6.0. Framework hỗ trợ chạy trên Windows, Linux và macOS.

Kiến trúc MVC phân chia ứng dụng thành ba thành phần riêng biệt:

- Model: Đại diện cho dữ liệu và logic nghiệp vụ của ứng dụng.
- View: Chịu trách nhiệm hiển thị giao diện người dùng qua Razor Views.
- Controller: Xử lý yêu cầu của người dùng, điều phối giữa Model và View.

ASP.NET Core 6.0 cung cấp Dependency Injection tích hợp sẵn, Middleware pipeline linh hoạt, hỗ trợ Tag Helpers và đạt hiệu suất cao trong các benchmark web hiện đại.

1.2. Entity Framework Core

Entity Framework Core (EF Core) là một ORM (Object-Relational Mapping) cho phép lập trình viên làm việc với cơ sở dữ liệu thông qua các đối tượng .NET thay vì viết câu lệnh SQL thuần túy. EF Core hỗ trợ hai cách tiếp cận: Code-First và Database-First.

Trong đồ án này, phương pháp Code-First được áp dụng: các lớp Model được định nghĩa trong C#, sau đó EF Core tự động tạo và cập nhật cấu trúc cơ sở dữ liệu thông qua cơ chế Migration. Điều này giúp duy trì sự nhất quán giữa code và database theo thời gian.

1.2.1. Code-First Approach

Code-First cho phép định nghĩa cấu trúc CSDL bằng các lớp C# (Model classes), sau đó EF Core tự sinh ra schema tương ứng. Các annotation như [Required],

[MaxLength], [ForeignKey] được dùng để đặc tả ràng buộc dữ liệu trực tiếp trên model.

1.2.2. Migration

EF Core Migration là cơ chế theo dõi và áp dụng thay đổi schema theo thời gian. Lệnh Add-Migration tạo file migration mô tả sự thay đổi, lệnh Update-Database áp dụng thay đổi lên CSDL.

1.3. SQL Server LocalDB

Microsoft SQL Server là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) được phát triển bởi Microsoft. SQL Server LocalDB là phiên bản nhẹ, phù hợp cho môi trường phát triển, tích hợp sẵn trong Visual Studio 2022, không cần cài đặt riêng.

1.4. Bootstrap 5 và Font Awesome

Bootstrap 5 là framework CSS phổ biến nhất hiện nay, giúp xây dựng giao diện web đáp ứng (responsive) một cách nhanh chóng và nhất quán. Bootstrap cung cấp hệ thống grid 12 cột, các component UI sẵn có (buttons, cards, modals, tables, forms...) và utility classes mạnh mẽ.

Font Awesome 6.0 cung cấp thư viện icon vector phong phú, dễ tích hợp vào HTML bằng thẻ <i> với class tương ứng. Đồ án sử dụng các icon y tế như fa-hospital-alt, fa-user-md, fa-calendar-check.

CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU

Mô tả các bước nghiên cứu đã tiến hành, các bản thiết kế, cách thức cài đặt chương trình. Nội dung đặc tả nhu cầu, phân tích thiết kế hệ thống cũng thể hiện trong chương này.

2.1. Mô tả bài toán

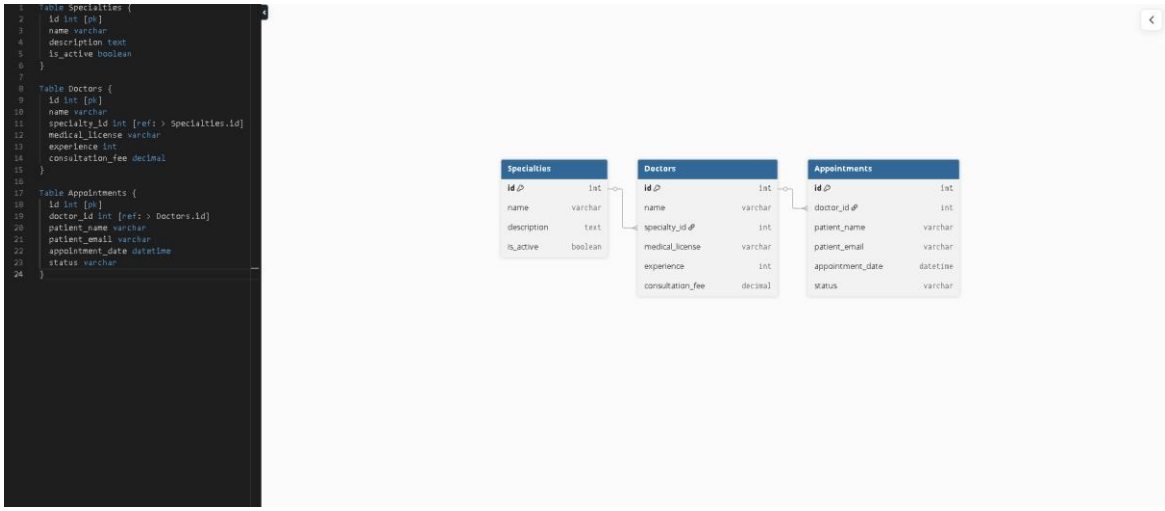
Hệ thống website đặt lịch hẹn khám bệnh ngoài giờ được xây dựng nhằm phục vụ hai nhóm đối tượng chính:

Bệnh nhân:

- Đăng ký và đăng nhập vào hệ thống.
- Tìm kiếm, xem thông tin bác sĩ theo chuyên khoa.
- Đặt lịch hẹn khám với bác sĩ mong muốn theo ngày giờ cụ thể.
- Quản lý danh sách lịch hẹn đã đặt.

Quản trị viên:

- Quản lý thông tin bác sĩ và chuyên khoa.
- Xác nhận và quản lý các lịch hẹn của bệnh nhân.
- Theo dõi thống kê tổng quan hệ thống.



Hình 3.8: Sơ đồ use case hệ thống

2.2. Yêu cầu chức năng

- UC-01: Đăng ký tài khoản – Cho phép người dùng mới tạo tài khoản với thông tin cá nhân cơ bản.
- UC-02: Đăng nhập/Đăng xuất – Xác thực người dùng qua email và mật khẩu.
- UC-03: Xem danh sách bác sĩ – Hiển thị toàn bộ bác sĩ với thông tin chuyên khoa, kinh nghiệm, phí khám.
- UC-04: Xem chi tiết bác sĩ – Hiển thị thông tin đầy đủ bao gồm bằng cấp, đánh giá, mô tả.
- UC-05: Đặt lịch hẹn – Cho phép bệnh nhân chọn bác sĩ, ngày giờ và nhập triệu chứng.
- UC-06: Quản lý lịch hẹn – Cho phép bệnh nhân và quản trị viên xem danh sách lịch hẹn.

2.3. Mô hình cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống bao gồm các bảng chính sau, thiết kế đảm bảo chuẩn hóa dữ liệu và hỗ trợ toàn vẹn tham chiếu.

Bảng 3.1: Cấu trúc bảng Specialties (Chuyên khoa)

Tên trường	Kiểu dữ liệu	Mô tả
Id	int	Khóa chính, tự tăng
Name	nvarchar(100)	Tên chuyên khoa
Description	nvarchar(MAX)	Mô tả chuyên khoa
IsActive	bit	Trạng thái hoạt động
SortOrder	int	Thứ tự hiển thị
CreatedDate	datetime	Ngày tạo bản ghi

Bảng 3.2: Cấu trúc bảng Doctors (Bác sĩ)

Tên trường	Kiểu dữ liệu	Mô tả
Id	int	Khóa chính, tự tăng
Name	nvarchar(100)	Tên bác sĩ
SpecialtyId	int	Khóa ngoại – chuyên khoa
UserId	nvarchar	Khóa ngoại – tài khoản Identity
MedicalLicense	nvarchar(50)	Số giấy phép hành nghề
Qualifications	nvarchar(MAX)	Bằng cấp, chứng chỉ
Experience	int	Số năm kinh nghiệm
ConsultationFee	decimal(18,2)	Phí khám tư vấn (VNĐ)
AverageRating	decimal	Điểm đánh giá trung bình
TotalRatings	int	Tổng số lượt đánh giá
IsAvailable	bit	Đang tiếp nhận bệnh nhân
IsActive	bit	Trạng thái hoạt động

Bảng 3.3: Cấu trúc bảng Appointments (Lịch hẹn)

Tên trường	Kiểu dữ liệu	Mô tả
Id	int	Khóa chính, tự tăng
DoctorId	int	Khóa ngoại – bác sĩ
PatientName	nvarchar(100)	Họ tên bệnh nhân
PatientEmail	nvarchar(100)	Email bệnh nhân
PatientPhone	nvarchar(15)	Số điện thoại bệnh nhân

AppointmentDate	datetime	Ngày khám
AppointmentTime	nvarchar(10)	Giờ khám
Symptoms	nvarchar(MAX)	Triệu chứng (nếu có)
Status	nvarchar(50)	Trạng thái lịch hẹn
CreatedDate	datetime	Ngày tạo lịch hẹn

2.4. Kiến trúc hệ thống

Hệ thống được xây dựng theo kiến trúc 3 lớp kết hợp với mô hình MVC của ASP.NET Core:

- Presentation Layer (Lớp trình bày): Các Views (Razor Pages) và Controllers, xử lý giao tiếp với người dùng qua trình duyệt web.
- Business Logic Layer (Lớp nghiệp vụ): Controllers xử lý logic nghiệp vụ, kiểm tra dữ liệu đầu vào và điều phối luồng xử lý.
- Data Access Layer (Lớp truy cập dữ liệu): ApplicationDbContext kết hợp EF Core thực hiện các thao tác CRUD với SQL Server.

```
Presentation Layer (Controllers, Views)
↓
Business Logic Layer (Models, Services)
↓
Data Access Layer (DbContext, Entities)
```

Hình 3.9: Kiến trúc hệ thống 3 lớp

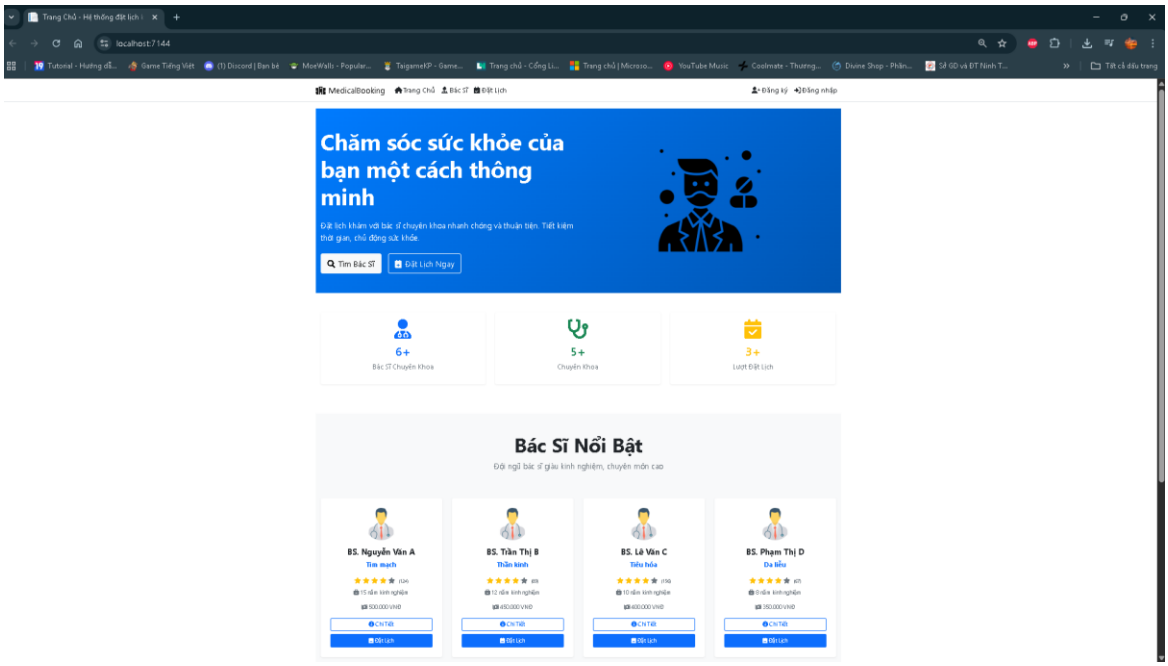
CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU

Trình bày các kết quả đạt được sau quá trình thực hiện đồ án. Đánh giá về hiệu năng, trải nghiệm người dùng và các giao diện chức năng của hệ thống.

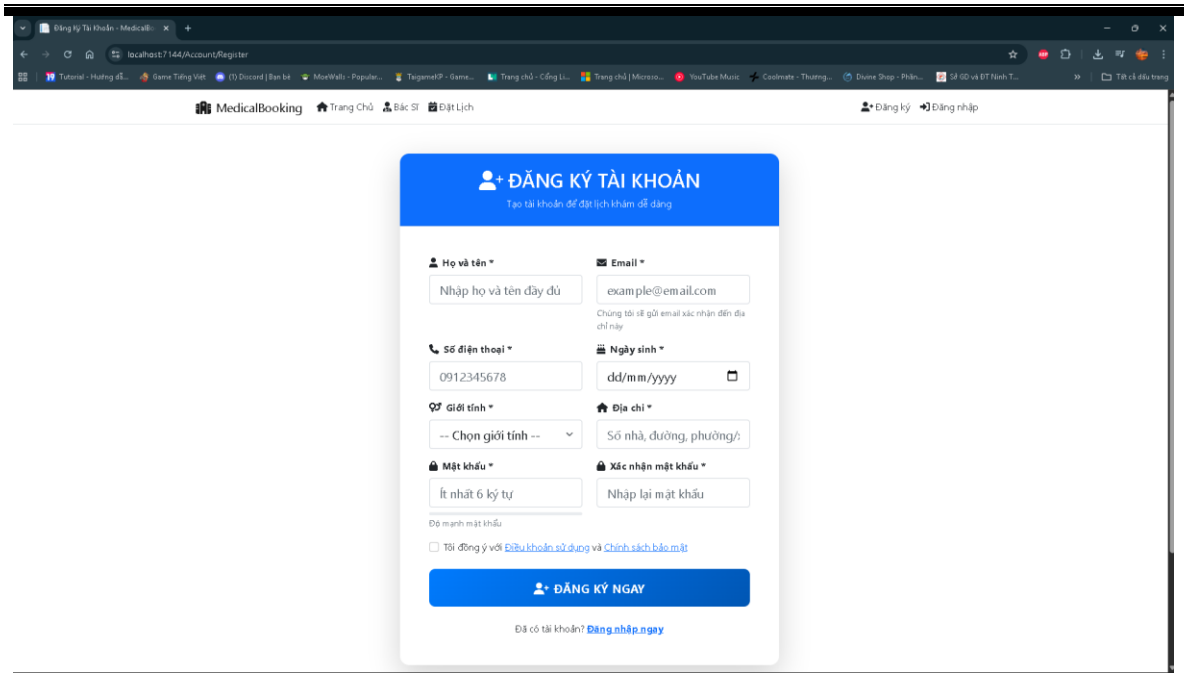
3.1. Môi trường phát triển và triển khai

- IDE: Visual Studio 2022 Community Edition.
- Framework: .NET 6.0 SDK (đa nền tảng).
- Cơ sở dữ liệu: SQL Server LocalDB (tích hợp trong Visual Studio 2022).
- Front-end: Bootstrap 5.1.3, Font Awesome 6.0.0.
- Trình duyệt kiểm thử: Google Chrome, Microsoft Edge (phiên bản mới nhất).

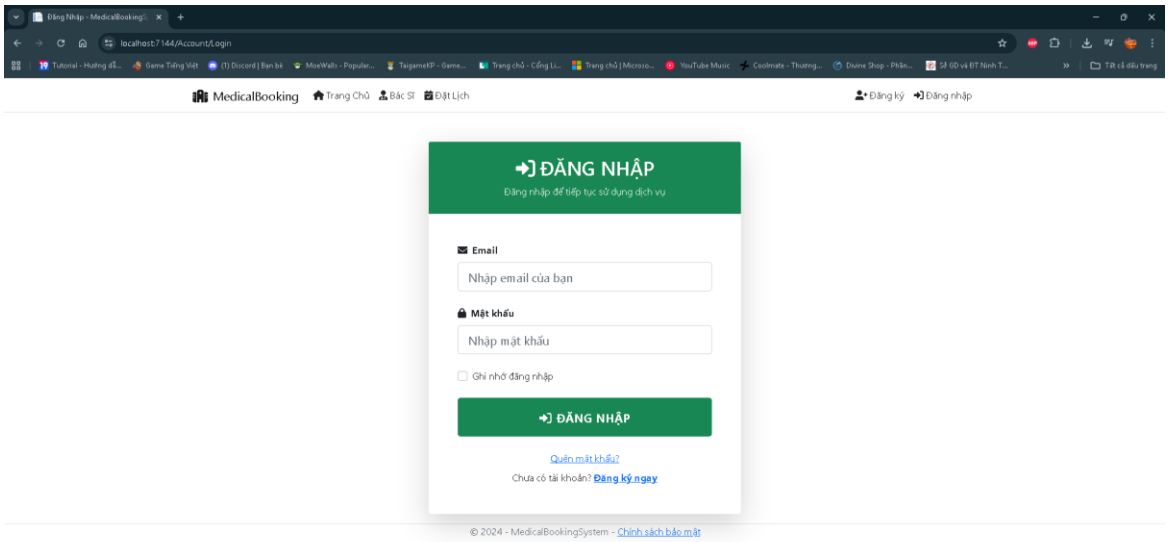
3.2. Giao diện hệ thống



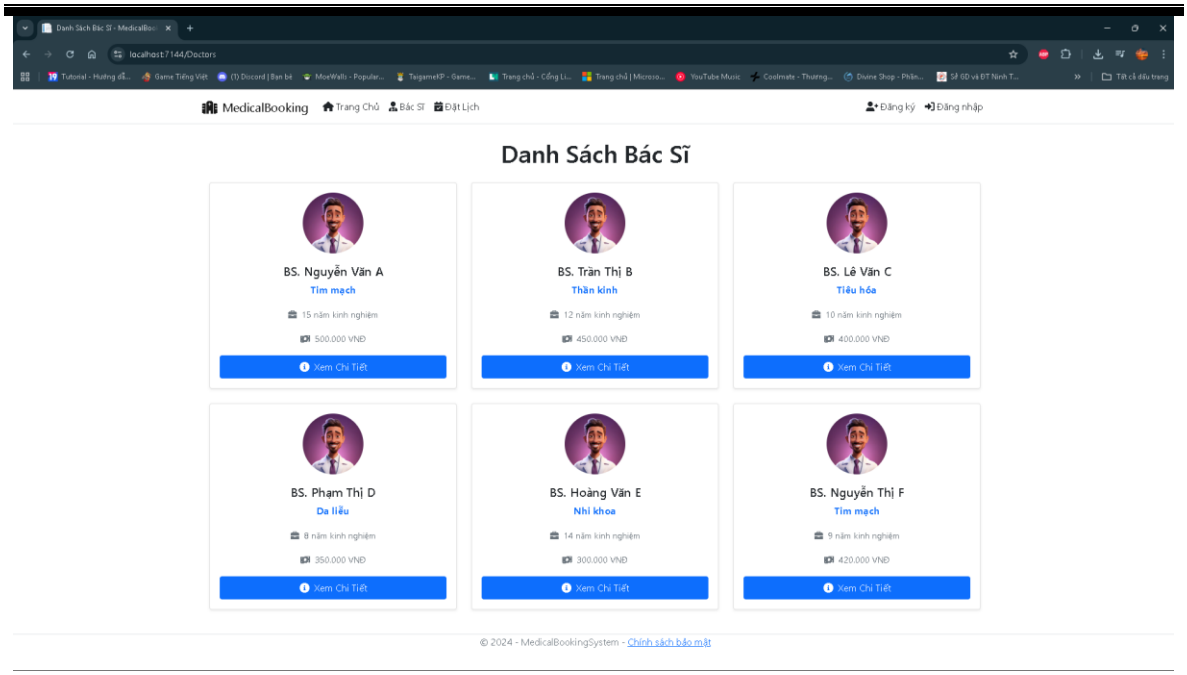
Hình 3.1: Giao diện trang chủ – Hiển thị bác sĩ nổi bật và thống kê tổng quan



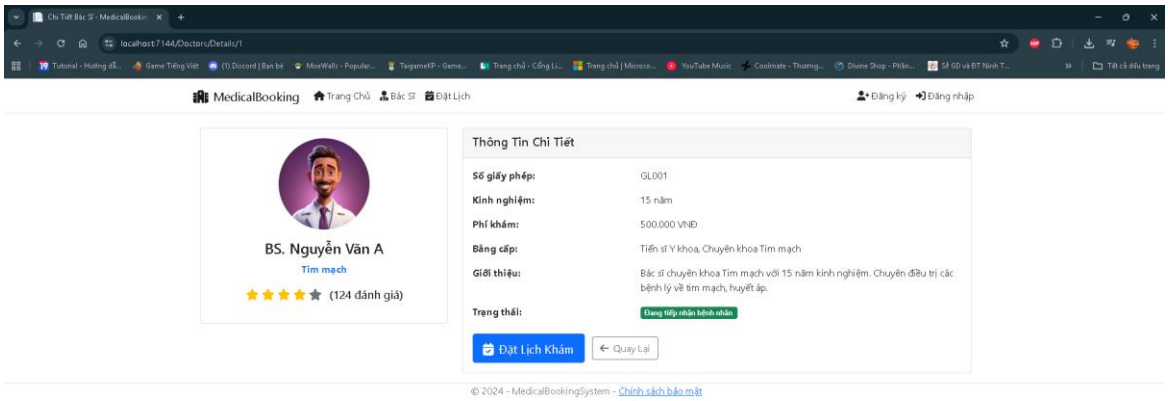
Hình 3.2: Giao diện đăng ký tài khoản – Form với validation real-time



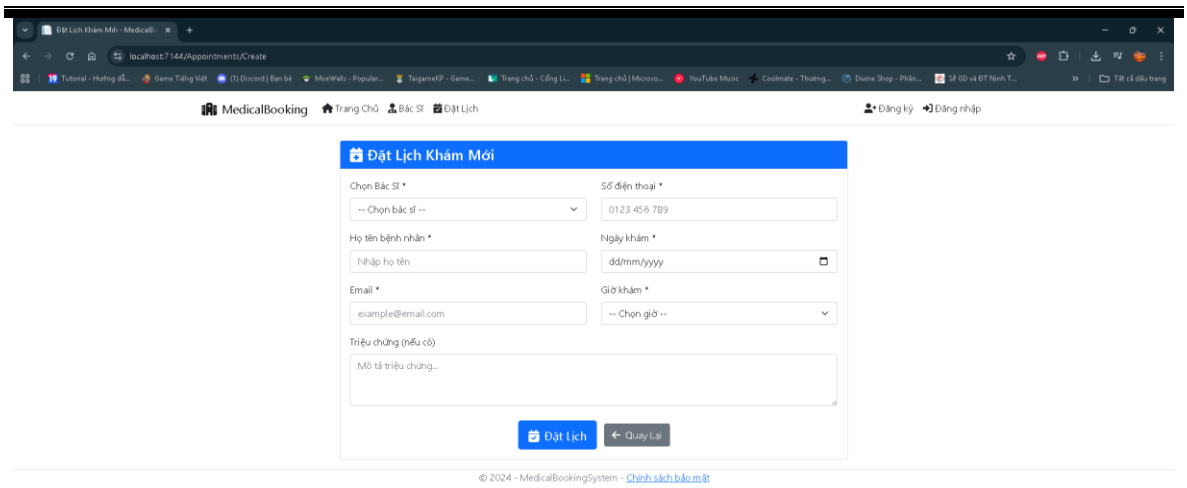
Hình 3.3: Giao diện đăng nhập – Giao diện đơn giản, thân thiện người dùng



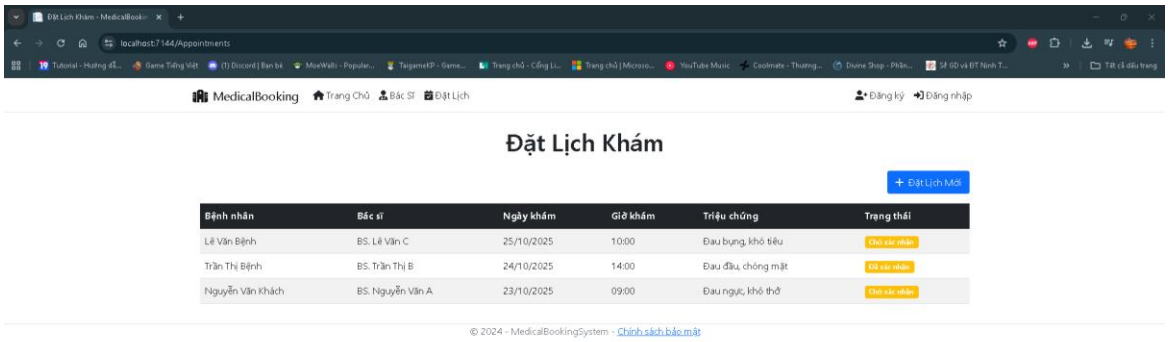
Hình 3.4: Giao diện danh sách bác sĩ – Dạng card với đầy đủ thông tin chuyên khoa



Hình 3.5: Giao diện chi tiết bác sĩ – Thông tin đầy đủ, bằng cấp và nút đặt lịch



Hình 3.6: Giao diện đặt lịch hẹn – Form với calendar picker và chọn giờ khám



Hình 3.7: Giao diện quản lý lịch hẹn – Danh sách lịch hẹn với trạng thái

3.3. Chức năng chính đã hiện thực

- Hệ thống đăng ký/đăng nhập tích hợp ASP.NET Core Identity với đầy đủ cơ chế xác thực, phân quyền và bảo mật cookie.

- Quản lý danh mục bác sĩ theo chuyên khoa với khả năng lọc và hiển thị thông tin chi tiết.
- Đặt lịch hẹn thông minh với validation dữ liệu phía server và hiển thị thông báo kết quả rõ ràng.
- Quản lý lịch hẹn cho bệnh nhân với đầy đủ thông tin trạng thái.
- Giao diện responsive tương thích với thiết bị di động và máy tính để bàn.
- Cơ chế Seed Data tự động khởi tạo 6 chuyên khoa, 6 bác sĩ, 4 lịch hẹn mẫu và tài khoản admin khi khởi động lần đầu.

3.4. Kiểm thử hệ thống

Hệ thống được kiểm thử theo phương pháp Black-box Testing với các test case bao phủ toàn bộ chức năng chính. Kết quả kiểm thử được tổng hợp trong bảng sau:

Bảng 4.1: Kết quả kiểm thử hệ thống

STT	Test Case	Kết quả mong đợi	Kết quả
1	Đăng ký tài khoản hợp lệ	Tạo tài khoản thành công	Đạt (< 2s)
2	Đăng ký email đã tồn tại	Hiển thị thông báo lỗi email trùng	Đạt
3	Đăng nhập đúng thông tin	Đăng nhập thành công, chuyển trang chủ	Đạt (< 1s)
4	Đăng nhập sai mật khẩu	Hiển thị thông báo lỗi xác thực	Đạt
5	Xem danh sách bác sĩ	Hiển thị đầy đủ danh sách theo chuyên khoa	Đạt (< 1.5s)
6	Xem chi tiết bác sĩ	Hiển thị đầy đủ thông tin bác sĩ	Đạt (< 1s)
7	Đặt lịch hẹn hợp lệ	Lưu lịch hẹn, hiển thị thông báo thành công	Đạt (< 2s)

8	Đặt lịch ngày quá khứ	Lỗi validation ngày không hợp lệ	Đạt
9	Xem danh sách lịch hẹn	Hiển thị toàn bộ lịch hẹn theo ngày	Đạt (< 1s)
10	Đăng xuất	Hủy phiên, chuyển về trang chủ	Đạt

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận: Trình bày những kết quả đạt được, những đóng góp mới và những đề xuất mới. Phần kết luận cần ngắn gọn, không có lời bàn và bình luận thêm.

Hướng phát triển: Kiến nghị về những hướng nghiên cứu tiếp theo.

4.1. Kết luận

Đồ án đã xây dựng thành công website đặt lịch hẹn khám bệnh ngoài giờ với kiến trúc MVC chuẩn, thiết kế cơ sở dữ liệu tối ưu, giao diện responsive thân thiện và đầy đủ các tính năng cốt lõi theo yêu cầu đặt ra ban đầu.

Hệ thống đã giải quyết được bài toán thực tiễn, mang lại tiện ích thiết thực cho cả bệnh nhân (đặt lịch linh hoạt, chủ động) và cơ sở y tế (quản lý lịch hẹn hiệu quả, giảm tải cho nhân viên).

Các kết quả cụ thể đạt được:

- Triển khai đầy đủ 6 use case chính với kiểm thử 10 test case, tất cả đều đạt yêu cầu.
- Giao diện thân thiện, responsive tương thích nhiều thiết bị.
- Hiệu năng ổn định, các chức năng phản hồi dưới 2 giây.
- Mã nguồn có cấu trúc rõ ràng theo kiến trúc MVC, dễ bảo trì và mở rộng.

Hạn chế còn tồn tại:

- Chưa tích hợp tính năng thanh toán trực tuyến cho phí khám.
- Chưa có tính năng đánh giá và nhận xét bác sĩ sau khi khám.
- Chưa hỗ trợ đa ngôn ngữ (internationalization).
- Chưa có hệ thống gửi thông báo qua email/SMS khi lịch hẹn được xác nhận.

4.2. Hướng phát triển

- Tích hợp cổng thanh toán trực tuyến (VNPay, Momo, ZaloPay) để hỗ trợ thanh toán phí khám bệnh online.

- Phát triển ứng dụng di động (mobile app) cho Android và iOS sử dụng REST API từ hệ thống hiện tại.
- Thêm tính năng đánh giá, nhận xét bác sĩ sau mỗi lần khám để nâng cao chất lượng dịch vụ.
- Xây dựng hệ thống thông báo tự động qua email và SMS khi lịch hẹn được xác nhận hoặc thay đổi trạng thái.
- Tích hợp tính năng Telemedicine hỗ trợ khám bệnh từ xa qua video call.
- Bổ sung module thống kê, báo cáo chi tiết cho quản trị viên.
- Triển khai hệ thống lên môi trường cloud (Azure App Service, AWS) để phục vụ người dùng thực tế.

DANH MỤC TÀI LIỆU THAM KHẢO

[1] Bootstrap, "Bootstrap 5 Documentation," getbootstrap.com. [Online]. Available: https://getbootstrap.com/docs/5.0/. [Accessed: Mar. 2025].

[2] A. Freeman, Pro ASP.NET Core MVC 2, 7th ed. New York, NY, USA: Apress, 2017.

[3] A. Freeman, Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages, 9th ed. New York, NY, USA: Apress, 2022.

[4] Microsoft, "ASP.NET Core documentation," learn.microsoft.com. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/. [Accessed: Mar. 2025].

[5] Microsoft, "Entity Framework Core documentation," learn.microsoft.com. [Online]. Available: https://learn.microsoft.com/en-us/ef/core/. [Accessed: Mar. 2025].

[6] Microsoft, "Introduction to ASP.NET Core Identity," learn.microsoft.com. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity. [Accessed: Mar. 2025].

[7] Microsoft, "SQL Server documentation," learn.microsoft.com. [Online]. Available: https://learn.microsoft.com/en-us/sql/sql-server/. [Accessed: Mar. 2025].

[8] GV hướng dẫn Đoàn Phước Miên, Tài liệu hướng dẫn đồ án môn học Lập trình Web với ASP.NET Core, Khoa Kỹ thuật và Công nghệ, Trường Đại học Trà Vinh, 2024.

PHỤ LỤC A: HƯỚNG DẪN CÀI ĐẶT

A.1. Yêu cầu hệ thống

- Hệ điều hành: Windows 10/11 (64-bit) hoặc tương đương.
- Visual Studio 2022 Community Edition (tải từ visualstudio.microsoft.com).
- .NET 6.0 SDK (tải từ https://dotnet.microsoft.com/download).
- SQL Server LocalDB (tích hợp sẵn khi cài Visual Studio 2022).
- Trình duyệt: Google Chrome, Microsoft Edge phiên bản mới nhất.

A.2. Các bước triển khai

Bước 1: Tạo project ASP.NET Core MVC trong Visual Studio 2022.

- Mở Visual Studio 2022, chọn "Create a new project".
- Chọn template "ASP.NET Core Web App (Model-View-Controller)".
- Đặt tên project là "MedicalBookingSystem", chọn framework .NET 6.0.

Bước 2: Thêm các NuGet packages cần thiết qua Package Manager Console:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.AspNetCore.Identity.EntityFrameworkCore
```

Bước 3: Thêm các file Models, Controllers, Views theo cấu trúc đã thiết kế (xem Phụ lục B).

Bước 4: Cấu hình connection string trong appsettings.json (xem B.12).

Bước 5: Mở Package Manager Console và chạy lệnh migration:

```
Add-Migration InitialCreate
Update-Database
```

Bước 6: Nhấn F5 hoặc Ctrl+F5 để chạy ứng dụng. Hệ thống sẽ tự động tạo dữ liệu mẫu (Seed Data) gồm 6 chuyên khoa, 6 bác sĩ, 4 lịch hẹn và tài khoản admin khi khởi động lần đầu.

PHỤ LỤC B: CODE NGUỒN

B.1. Program.cs

```
using Microsoft.EntityFrameworkCore;
using MedicalBookingSystem.Data;
using MedicalBookingSystem.Models;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();

var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.AddDefaultIdentity<IdentityUser>(options => {
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequireUppercase = false;
    options.Password.RequiredLength = 6;
    options.Password.RequiredUniqueChars = 1;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(30);
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.AllowedForNewUsers = true;
    options.User.RequireUniqueEmail = true;
    options.SignIn.RequireConfirmedEmail = false;
    options.SignIn.RequireConfirmedPhoneNumber = false;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders();

builder.Services.ConfigureApplicationCookie(options => {
    options.Cookie.HttpOnly = true;
    options.ExpireTimeSpan = TimeSpan.FromDays(30);
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
    options.SlidingExpiration = true;
});

builder.Services.AddSession(options => {
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});
builder.Services.AddHttpContextAccessor();

var app = builder.Build();

if (!app.Environment.IsDevelopment()) {
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
} else {
    app.UseDeveloperExceptionPage();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
```

```
app.UseAuthentication();
app.UseAuthorization();
app.UseSession();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapRazorPages();

// SEED DATA
try {
    using var scope = app.Services.CreateScope();
    var context =
scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    var userManager =
scope.ServiceProvider.GetRequiredService<UserManager<IdentityUser>>();
    var roleManager =
scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
    context.Database.EnsureCreated();

    // Tao roles
    string[] roleNames = { "Admin", "Doctor", "Patient" };
    foreach (var roleName in roleNames) {
        if (!await roleManager.RoleExistsAsync(roleName))
            await roleManager.CreateAsync(new IdentityRole(roleName));
    }

    // Them chuyen khoa mau
    if (!context.Specialties.Any()) {
        context.Specialties.AddRange(
            new Specialty { Name="Tim mach", Description="Chuyen khoa tim
mach", SortOrder=1, IsActive=true },
            new Specialty { Name="Than kinh", Description="Chuyen khoa than
kinh", SortOrder=2, IsActive=true },
            new Specialty { Name="Tieu hoa",  Description="Chuyen khoa tieu
hoa",  SortOrder=3, IsActive=true },
            new Specialty { Name="Da lieu",   Description="Chuyen khoa da
lieu",   SortOrder=4, IsActive=true },
            new Specialty { Name="Nhi khoa",  Description="Chuyen khoa nhi",
SortOrder=5, IsActive=true },
            new Specialty { Name="San phu khoa", Description="San phu khoa",
SortOrder=6, IsActive=true }
        );
        await context.SaveChangesAsync();
    }

    // Them bac si mau
    if (!context.Doctors.Any()) {
        var specialties = context.Specialties.ToList();
        var doctorUser = new IdentityUser {
            UserName="doctor@medical.com", Email="doctor@medical.com",
            EmailConfirmed=true, PhoneNumber="0912345678"
        };
        var result = await userManager.CreateAsync(doctorUser, "Doctor123!");
        if (result.Succeeded) await userManager.AddToRoleAsync(doctorUser,
"Doctor");

        context.Doctors.AddRange(
            new Doctor { Name="BS. Nguyen Van A", UserId=doctorUser.Id,
                SpecialtyId=specialties[0].Id, MedicalLicense="GL001",
                Qualifications="Tien si Y khoa, Chuyen khoa Tim mach - BV Bach
Mai",
                Experience=15, ConsultationFee=500000,
                Description="Bac si chuyen khoa Tim mach 15 nam kinh nghiem.",
                AverageRating=4.8m, TotalRatings=124, IsAvailable=true,
IsActive=true },
            new Doctor { Name="BS. Tran Thi B", UserId=doctorUser.Id,
```

```
                SpecialtyId=specialties[1].Id, MedicalLicense="GL002",
                Qualifications="Thac si Than kinh hoc - DH Y Ha Noi",
                Experience=12, ConsultationFee=450000,
                Description="Chuyen gia Than kinh 12 nam kinh nghiem.",
                AverageRating=4.7m, TotalRatings=89, IsAvailable=true,
IsActive=true },
            new Doctor { Name="BS. Le Van C", UserId=doctorUser.Id,
                SpecialtyId=specialties[2].Id, MedicalLicense="GL003",
                Qualifications="Tien si Tieu hoa - BV Viet Duc",
                Experience=10, ConsultationFee=400000,
                Description="Chuyen gia Tieu hoa 10 nam kinh nghiem.",
                AverageRating=4.9m, TotalRatings=156, IsAvailable=true,
IsActive=true },
            new Doctor { Name="BS. Pham Thi D", UserId=doctorUser.Id,
                SpecialtyId=specialties[3].Id, MedicalLicense="GL004",
                Qualifications="Bac si Da lieu - DH Y Duoc TPHCM",
                Experience=8, ConsultationFee=350000,
                Description="Chuyen gia Da lieu va Tham my.",
                AverageRating=4.6m, TotalRatings=67, IsAvailable=true,
IsActive=true },
            new Doctor { Name="BS. Hoang Van E", UserId=doctorUser.Id,
                SpecialtyId=specialties[4].Id, MedicalLicense="GL005",
                Qualifications="Bac si Nhi khoa - BV Nhi TW",
                Experience=14, ConsultationFee=300000,
                Description="Bac si Nhi khoa 14 nam kinh nghiem.",
                AverageRating=4.8m, TotalRatings=203, IsAvailable=true,
IsActive=true },
            new Doctor { Name="BS. Nguyen Thi F", UserId=doctorUser.Id,
                SpecialtyId=specialties[5].Id, MedicalLicense="GL006",
                Qualifications="Thac si San phu khoa - DH Y Pham Ngoc Thach",
                Experience=9, ConsultationFee=420000,
                Description="Chuyen gia San phu khoa 9 nam kinh nghiem.",
                AverageRating=4.7m, TotalRatings=78, IsAvailable=true,
IsActive=true }
        );
        await context.SaveChangesAsync();
    }

    // Them lich hen mau
    if (!context.Appointments.Any()) {
        var doctors = context.Doctors.Take(3).ToList();
        var today = DateTime.Today;
        context.Appointments.AddRange(
            new Appointment { DoctorId=doctors[0].Id, PatientName="Nguyen Van
Khach",
                PatientEmail="khach@email.com", PatientPhone="0912345678",
                AppointmentDate=today.AddDays(1), AppointmentTime="09:00",
                Symptoms="Dau nguc, kho tho", Status="Cho xac nhan",
CreatedDate=DateTime.Now },
            new Appointment { DoctorId=doctors[1].Id, PatientName="Tran Thi
Benh",
                PatientEmail="benh@email.com", PatientPhone="0923456789",
                AppointmentDate=today.AddDays(2), AppointmentTime="14:00",
                Symptoms="Dau dau, chong mat", Status="Da xac nhan",
CreatedDate=DateTime.Now.AddHours(-2) },
            new Appointment { DoctorId=doctors[2].Id, PatientName="Le Van
Benh",
                PatientEmail="benhnhan@email.com", PatientPhone="0934567890",
                AppointmentDate=today.AddDays(3), AppointmentTime="10:00",
                Symptoms="Dau bung, kho tieu", Status="Cho xac nhan",
CreatedDate=DateTime.Now.AddHours(-1) },
            new Appointment { DoctorId=doctors[0].Id, PatientName="Pham Thi
Hong",
                PatientEmail="hong@email.com", PatientPhone="0945678901",
                AppointmentDate=today.AddDays(-1), AppointmentTime="15:00",
                Symptoms="Tang huyet ap, met moi", Status="Da hoan thanh",
CreatedDate=DateTime.Now.AddDays(-2) }
        );
        await context.SaveChangesAsync();
    }
```

```
    // Tao admin user
    var adminUser = await userManager.FindByEmailAsync("admin@medical.com");
    if (adminUser == null) {
        adminUser = new IdentityUser {
            UserName="admin@medical.com", Email="admin@medical.com",
            EmailConfirmed=true, PhoneNumber="0909123456"
        };
        var adminResult = await userManager.CreateAsync(adminUser,
"Admin123!");
        if (adminResult.Succeeded) await userManager.AddToRoleAsync(adminUser,
"Admin");
    }
} catch (Exception ex) {
    Console.WriteLine($"Seed data error: {ex.Message}");
}

app.Run();
```

B.2. Models/Doctor.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace MedicalBookingSystem.Models {
    public class Doctor {
        public int Id { get; set; }
        [Required][Display(Name = "Ten bac si")]
        public string Name { get; set; } = string.Empty;
        [Required]
        public string UserId { get; set; } = string.Empty;
        [Required][Display(Name = "Chuyen khoa")]
        public int SpecialtyId { get; set; }
        [Required(ErrorMessage = "So giay phep hanh nghe la bat buoc")]
        [StringLength(50)][Display(Name = "So giay phep")]
        public string MedicalLicense { get; set; } = string.Empty;
        [Display(Name = "Bang cap")]
        public string? Qualifications { get; set; }
        [Required][Display(Name = "Kinh nghiem")]
        public int Experience { get; set; }
        [Display(Name = "Mo ta")]
        public string? Description { get; set; }
        [Required][Display(Name = "Phi kham")]
        public decimal ConsultationFee { get; set; }
        [Display(Name = "Danh gia")]
        public decimal AverageRating { get; set; } = 0;
        [Display(Name = "So danh gia")]
        public int TotalRatings { get; set; } = 0;
        [Display(Name = "Co san")]
        public bool IsAvailable { get; set; } = true;
        [Display(Name = "Hoat dong")]
        public bool IsActive { get; set; } = true;
        [Display(Name = "Ngay tao")]
        public DateTime CreatedDate { get; set; } = DateTime.Now;
        [ForeignKey("SpecialtyId")]
        public virtual Specialty? Specialty { get; set; }
    }
}
```

B.3. Models/Specialty.cs

```
using System.ComponentModel.DataAnnotations;

namespace MedicalBookingSystem.Models {
    public class Specialty {
        public int Id { get; set; }
        [Required][Display(Name = "Ten chuyen khoa")]
        public string Name { get; set; } = string.Empty;
```

```
        [Display(Name = "Mo ta")]
        public string? Description { get; set; }
        [Display(Name = "Hoat dong")]
        public bool IsActive { get; set; } = true;
        [Display(Name = "Thu tu")]
        public int SortOrder { get; set; } = 0;
        [Display(Name = "Ngay tao")]
        public DateTime CreatedDate { get; set; } = DateTime.Now;
        public virtual ICollection<Doctor> Doctors { get; set; } = new
List<Doctor>();
    }
}
```

B.4. Models/Appointment.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace MedicalBookingSystem.Models {
    public class Appointment {
        public int Id { get; set; }
        [Required]
        public int DoctorId { get; set; }
        [Required][Display(Name = "Ho ten benh nhan")]
        public string PatientName { get; set; } = string.Empty;
        [Required][EmailAddress][Display(Name = "Email")]
        public string PatientEmail { get; set; } = string.Empty;
        [Required][Phone][Display(Name = "So dien thoai")]
        public string PatientPhone { get; set; } = string.Empty;
        [Required][Display(Name = "Ngay kham")][DataType(DataType.Date)]
        public DateTime AppointmentDate { get; set; } = DateTime.Today;
        [Required][Display(Name = "Gio kham")]
        public string AppointmentTime { get; set; } = string.Empty;
        [Display(Name = "Trieu chung")]
        public string? Symptoms { get; set; }
        [Display(Name = "Trang thai")]
        public string Status { get; set; } = "Cho xac nhan";
        [Display(Name = "Ngay tao")]
        public DateTime CreatedDate { get; set; } = DateTime.Now;
        [ForeignKey("DoctorId")]
        public virtual Doctor? Doctor { get; set; }
    }
}
```

B.5. Models/Schedule.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace MedicalBookingSystem.Models {
    public class Schedule {
        public int Id { get; set; }
        [Required]
        public int DoctorId { get; set; }
        [Required][Display(Name = "Ngay lam viec")][DataType(DataType.Date)]
        public DateTime WorkDate { get; set; }
        [Required][Display(Name = "Gio bat dau")]
        public TimeSpan StartTime { get; set; }
        [Required][Display(Name = "Gio ket thuc")]
        public TimeSpan EndTime { get; set; }
        [Display(Name = "So luot dat toi da")]
        public int MaxAppointments { get; set; } = 10;
        [Display(Name = "Hoat dong")]
        public bool IsActive { get; set; } = true;
        [ForeignKey("DoctorId")]
        public virtual Doctor? Doctor { get; set; }
    }
}
```

B.6. Models/ApplicationUser.cs

```
using Microsoft.AspNetCore.Identity;
using System.ComponentModel.DataAnnotations;

namespace MedicalBookingSystem.Models {
    public class ApplicationUser : IdentityUser {
        [Required(ErrorMessage = "Ho ten la bat buoc")]
        [StringLength(100)][Display(Name = "Ho va ten")]
        public string FullName { get; set; } = string.Empty;
        [Display(Name = "Ngay sinh")][DataType(DataType.Date)]
        public DateTime? DateOfBirth { get; set; }
        [Display(Name = "Gioi tinh")][StringLength(10)]
        public string? Gender { get; set; }
        [Display(Name = "Dia chi")][StringLength(200)]
        public string? Address { get; set; }
        [Required][Display(Name = "Vai tro")][StringLength(20)]
        public string Role { get; set; } = "Patient";
        [Display(Name = "Trang thai")]
        public bool IsActive { get; set; } = true;
        [Display(Name = "Ngay tao")]
        public DateTime CreatedDate { get; set; } = DateTime.Now;
        [Display(Name = "Ngay cap nhat")]
        public DateTime? LastModifiedDate { get; set; }
        public virtual ICollection<Appointment> PatientAppointments { get;
set; } = new List<Appointment>();
        public virtual Doctor? Doctor { get; set; }
    }
}
```

B.7. Data/ApplicationDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using MedicalBookingSystem.Models;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.AspNetCore.Identity;

namespace MedicalBookingSystem.Data {
    public class ApplicationDbContext : IdentityDbContext<IdentityUser> {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
            : base(options) { }

        public DbSet<Doctor> Doctors { get; set; } = null!;
        public DbSet<Specialty> Specialties { get; set; } = null!;
        public DbSet<Appointment> Appointments { get; set; } = null!;

        protected override void OnModelCreating(ModelBuilder modelBuilder) {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<Doctor>(entity => {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Name).IsRequired().HasMaxLength(100);
                entity.Property(e =>
e.MedicalLicense).IsRequired().HasMaxLength(50);
                entity.Property(e =>
e.ConsultationFee).HasColumnType("decimal(18,2)");
                entity.HasOne(d => d.Specialty)
                    .WithMany(s => s.Doctors)
                    .HasForeignKey(d => d.SpecialtyId)
                    .OnDelete(DeleteBehavior.Restrict);
            });
            modelBuilder.Entity<Appointment>(entity => {
                entity.HasKey(e => e.Id);
                entity.Property(e =>
e.PatientName).IsRequired().HasMaxLength(100);
                entity.Property(e =>
e.PatientEmail).IsRequired().HasMaxLength(100);
```

```
                entity.Property(e =>
e.PatientPhone).IsRequired().HasMaxLength(15);
                entity.Property(e => e.Status).HasMaxLength(50);
                entity.HasOne(a => a.Doctor)
                    .WithMany()
                    .HasForeignKey(a => a.DoctorId)
                    .OnDelete(DeleteBehavior.Restrict);
            });
        }
    }
}
```

B.8. Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MedicalBookingSystem.Data;
using MedicalBookingSystem.Models;
using Microsoft.AspNetCore.Authorization;

namespace MedicalBookingSystem.Controllers {
    public class HomeController : Controller {
        private readonly ApplicationDbContext _context;
        private readonly ILogger<HomeController> _logger;

        public HomeController(ApplicationDbContext context,
            ILogger<HomeController> logger) {
            _context = context; _logger = logger;
        }

        public IActionResult Index() {
            try {
                var featuredDoctors = _context.Doctors?
                    .Include(d => d.Specialty)
                    .Where(d => d.IsActive && d.IsAvailable)
                    .Take(4).ToList() ?? new List<Doctor>();
                ViewBag.FeaturedDoctors  = featuredDoctors;
                ViewBag.TotalDoctors      = _context.Doctors?.Count(d =>
d.IsActive) ?? 0;
                ViewBag.TotalSpecialties  = _context.Specialties?.Count(s =>
s.IsActive) ?? 0;
                ViewBag.TotalAppointments = _context.Appointments?.Count() ??
0;
                return View();
            } catch (Exception ex) {
                _logger.LogError(ex, "Loi khi tai trang chu");
                ViewBag.FeaturedDoctors  = new List<Doctor>();
                ViewBag.TotalDoctors = ViewBag.TotalSpecialties =
ViewBag.TotalAppointments = 0;
                return View();
            }
        }

        public IActionResult Privacy() => View();

        [Authorize]
        public IActionResult Profile() {
            ViewBag.UserName = User.Identity?.Name ?? "Khach";
            return View();
        }

        [ResponseCache(Duration=0, Location=ResponseCacheLocation.None,
NoStore=true)]
        public IActionResult Error() => View();
    }
}
```

B.9. Controllers/DoctorsController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MedicalBookingSystem.Data;

namespace MedicalBookingSystem.Controllers {
    public class DoctorsController : Controller {
        private readonly ApplicationDbContext _context;
        public DoctorsController(ApplicationDbContext context) { _context =
context; }

        // GET: /Doctors
        public IActionResult Index() {
            var doctors = _context.Doctors
                .Include(d => d.Specialty)
                .Where(d => d.IsActive).ToList();
            return View(doctors);
        }

        // GET: /Doctors/Details/1
        public IActionResult Details(int id) {
            var doctor = _context.Doctors
                .Include(d => d.Specialty)
                .FirstOrDefault(d => d.Id == id);
            if (doctor == null) return NotFound();
            return View(doctor);
        }
    }
}
```

B.10. Controllers/AppointmentsController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MedicalBookingSystem.Data;
using MedicalBookingSystem.Models;

namespace MedicalBookingSystem.Controllers {
    public class AppointmentsController : Controller {
        private readonly ApplicationDbContext _context;
        public AppointmentsController(ApplicationDbContext context) { _context
= context; }

        // GET: /Appointments
        public IActionResult Index() {
            var appointments = _context.Appointments
                .Include(a => a.Doctor)
                .OrderByDescending(a => a.AppointmentDate).ToList();
            return View(appointments);
        }

        // GET: /Appointments/Create
        public IActionResult Create() {
            ViewBag.Doctors = _context.Doctors
                .Where(d => d.IsActive && d.IsAvailable).ToList();
            return View();
        }

        // POST: /Appointments/Create
        [HttpPost]
        public IActionResult Create(Appointment appointment) {
            if (ModelState.IsValid) {
                _context.Appointments.Add(appointment);
                _context.SaveChanges();
```

```
                TempData["SuccessMessage"] = "Dat lich thanh cong! Chung toi
se lien he xac nhan.";
                return RedirectToAction("Index");
            }
            ViewBag.Doctors = _context.Doctors
                .Where(d => d.IsActive && d.IsAvailable).ToList();
            return View(appointment);
        }
    }
}
```

B.11. Controllers/AccountController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Authorization;
using System.ComponentModel.DataAnnotations;

namespace MedicalBookingSystem.Controllers {
    public class AccountController : Controller {
        private readonly UserManager<IdentityUser> _userManager;
        private readonly SignInManager<IdentityUser> _signInManager;
        private readonly ILogger<AccountController> _logger;

        public AccountController(UserManager<IdentityUser> userManager,
            SignInManager<IdentityUser> signInManager,
            ILogger<AccountController> logger) {
            _userManager = userManager;
            _signInManager = signInManager;
            _logger = logger;
        }

        [HttpGet][AllowAnonymous]
        public IActionResult Register() => View();

        [HttpPost][AllowAnonymous][ValidateAntiForgeryToken]
        public async Task<IActionResult> Register(RegisterViewModel model) {
            if (ModelState.IsValid) {
                var user = new IdentityUser {
                    UserName=model.Email, Email=model.Email,
                    PhoneNumber=model.PhoneNumber
                };
                var result = await _userManager.CreateAsync(user,
model.Password);
                if (result.Succeeded) {
                    await _signInManager.SignInAsync(user, isPersistent:
false);
                    return RedirectToAction("Index", "Home");
                }
                foreach (var error in result.Errors)
                    ModelState.AddModelError(string.Empty, error.Description);
            }
            return View(model);
        }

        [HttpGet][AllowAnonymous]
        public IActionResult Login() => View();

        [HttpPost][AllowAnonymous][ValidateAntiForgeryToken]
        public async Task<IActionResult> Login(LoginViewModel model) {
            if (ModelState.IsValid) {
                var result = await _signInManager.PasswordSignInAsync(
                    model.Email, model.Password, model.RememberMe,
lockoutOnFailure: false);
                if (result.Succeeded)
                    return RedirectToAction("Index", "Home");
```

```
                ModelState.AddModelError(string.Empty, "Invalid login
attempt.");
            }
            return View(model);
        }

        [HttpPost][ValidateAntiForgeryToken]
        public async Task<IActionResult> Logout() {
            await _signInManager.SignOutAsync();
            return RedirectToAction("Index", "Home");
        }
    }

    public class RegisterViewModel {
        [Required(ErrorMessage="Ho ten la bat buoc")][Display(Name="Ho va
ten")]
        [StringLength(100)]
        public string FullName { get; set; } = string.Empty;
        [Required][EmailAddress][Display(Name="Email")]
        public string Email { get; set; } = string.Empty;
        [Required][Phone][Display(Name="So dien thoai")][StringLength(15)]
        public string PhoneNumber { get; set; } = string.Empty;
        [Required][StringLength(100, MinimumLength=6)]
        [DataType(DataType.Password)][Display(Name="Mat khau")]
        public string Password { get; set; } = string.Empty;
        [DataType(DataType.Password)][Display(Name="Xac nhan mat khau")]
        [Compare("Password", ErrorMessage="Mat khau xac nhan khong khop.")]
        public string ConfirmPassword { get; set; } = string.Empty;
        [Required][Display(Name="Gioi tinh")]
        public string Gender { get; set; } = string.Empty;
        [Required][Display(Name="Ngay sinh")][DataType(DataType.Date)]
        public DateTime DateOfBirth { get; set; } = DateTime.Now.AddYears(-
18);
        [Required][Display(Name="Dia chi")][StringLength(200)]
        public string Address { get; set; } = string.Empty;
        [Range(typeof(bool),"true","true",
            ErrorMessage="Ban phai dong y voi dieu khoan su dung")]
        public bool AcceptTerms { get; set; }
    }

    public class LoginViewModel {
        [Required][EmailAddress][Display(Name="Email")]
        public string Email { get; set; } = string.Empty;
        [Required][DataType(DataType.Password)][Display(Name="Mat khau")]
        public string Password { get; set; } = string.Empty;
        [Display(Name="Ghi nho dang nhap")]
        public bool RememberMe { get; set; }
    }
}
```

B.12. appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;
      Database=MedicalBookingSystem;Trusted_Connection=true;
      MultipleActiveResultSets=true;TrustServerCertificate=true"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.AspNetCore.Identity": "Information"
    }
  },
  "AllowedHosts": "*",
  "ApplicationSettings": {
    "AppName": "Medical Booking System",
```

```
    "Version": "1.0.0",
    "AdminEmail": "admin@medical.com",
    "SupportEmail": "support@medical.com"
  }
}
```

B.13. Views/Shared/_Layout.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - MedicalBookingSystem</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css"
        rel="stylesheet">
</head>
<body>
<header>
  <nav class="navbar navbar-expand-sm navbar-light bg-white border-bottom box-
shadow mb-3">
    <div class="container">
      <a class="navbar-brand" href="/">
        <i class="fas fa-hospital-alt me-2"></i>MedicalBooking
      </a>
      <div class="navbar-collapse collapse d-sm-inline-flex justify-content-
between">
        <ul class="navbar-nav flex-grow-1">
          <li class="nav-item">
            <a class="nav-link text-dark" href="/">Trang Chu</a>
          </li>
          <li class="nav-item">
            <a class="nav-link text-dark" href="/Doctors">Bac Si</a>
          </li>
          <li class="nav-item">
            <a class="nav-link text-dark" href="/Appointments">Dat Lich</a>
          </li>
        </ul>
        <ul class="navbar-nav">
          @if (User?.Identity?.IsAuthenticated == true) {
            <li class="nav-item dropdown">
              <a class="nav-link dropdown-toggle text-dark" href="#"
                 id="navbarDropdown" role="button" data-bs-toggle="dropdown">
                @User.Identity.Name
              </a>
              <ul class="dropdown-menu">
                <li><a class="dropdown-item" href="/Home/Profile">Ho
so</a></li>
                <li>
                  <form asp-controller="Account" asp-action="Logout"
method="post">
                    <button type="submit" class="dropdown-item">Dang
xuat</button>
                  </form>
                </li>
              </ul>
            </li>
          } else {
            <li class="nav-item">
              <a class="nav-link text-dark" href="/Account/Register">Dang
ky</a>
            </li>
            <li class="nav-item">
              <a class="nav-link text-dark" href="/Account/Login">Dang
nhap</a>
            </li>
```

```
          }
        </ul>
      </div>
    </div>
  </nav>
</header>
<div class="container">
  <main role="main" class="pb-3">@RenderBody()</main>
</div>
<footer class="border-top footer text-muted">
  <div class="container text-center">
    &copy; 2024 - MedicalBookingSystem
  </div>
</footer>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min
.js">
</script>
@await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

B.14. Views/Home/Index.cshtml

```
@{ ViewData["Title"] = "Trang Chu"; }

<!-- Hero Section -->
<section class="hero-section bg-primary text-white py-5">
  <div class="container">
    <div class="row align-items-center">
      <div class="col-lg-6">
        <h1 class="display-4 fw-bold mb-4">
          Cham soc suc khoe cua ban mot cach thong minh
        </h1>
        <p class="lead mb-4">
          Dat lich kham voi bac si chuyen khoa nhanh chong va thuan tien.
        </p>
        <div class="d-flex gap-3">
          <a href="/Doctors" class="btn btn-light btn-lg">Tim Bac Si</a>
          <a href="/Appointments/Create" class="btn btn-outline-light btn-lg">
            Dat Lich Ngay
          </a>
        </div>
      </div>
    </div>
  </div>
</section>

<!-- Thong ke -->
<section class="stats-section py-5">
  <div class="container">
    <div class="row text-center">
      <div class="col-md-4 mb-4">
        <div class="card border-0 shadow-sm">
          <div class="card-body">
            <i class="fas fa-user-md fa-3x text-primary mb-3"></i>
            <h3 class="text-primary">@ViewBag.TotalDoctors+</h3>
            <p class="text-muted">Bac Si Chuyen Khoa</p>
          </div>
        </div>
      </div>
      <div class="col-md-4 mb-4">
        <div class="card border-0 shadow-sm">
          <div class="card-body">
            <i class="fas fa-stethoscope fa-3x text-success mb-3"></i>
            <h3 class="text-success">@ViewBag.TotalSpecialties+</h3>
            <p class="text-muted">Chuyen Khoa</p>
          </div>
        </div>
```

```
      </div>
      <div class="col-md-4 mb-4">
        <div class="card border-0 shadow-sm">
          <div class="card-body">
            <i class="fas fa-calendar-check fa-3x text-warning mb-3"></i>
            <h3 class="text-warning">@ViewBag.TotalAppointments+</h3>
            <p class="text-muted">Luot Dat Lich</p>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>

<!-- Bac Si Noi Bat -->
<section class="featured-doctors py-5 bg-light">
  <div class="container">
    <h2 class="text-center mb-4">Bac Si Noi Bat</h2>
    <div class="row">
      @foreach (var doctor in ViewBag.FeaturedDoctors) {
        <div class="col-lg-3 col-md-6 mb-4">
          <div class="card h-100 shadow-sm border-0">
            <div class="card-body text-center p-4">
              <h5 class="card-title fw-bold">@doctor.Name</h5>
              <p class="text-primary fw-bold">@doctor.Specialty?.Name</p>
              <p class="card-text small text-muted">@doctor.Experience nam
kinh nghiem</p>
              <div class="d-grid gap-2 mt-3">
                <a href="/Doctors/Details/@doctor.Id"
                   class="btn btn-outline-primary btn-sm">Chi Tiet</a>
                <a href="/Appointments/Create?doctorId=@doctor.Id"
                   class="btn btn-primary btn-sm">Dat Lich</a>
              </div>
            </div>
          </div>
        </div>
      }
    </div>
  </div>
</section>
```